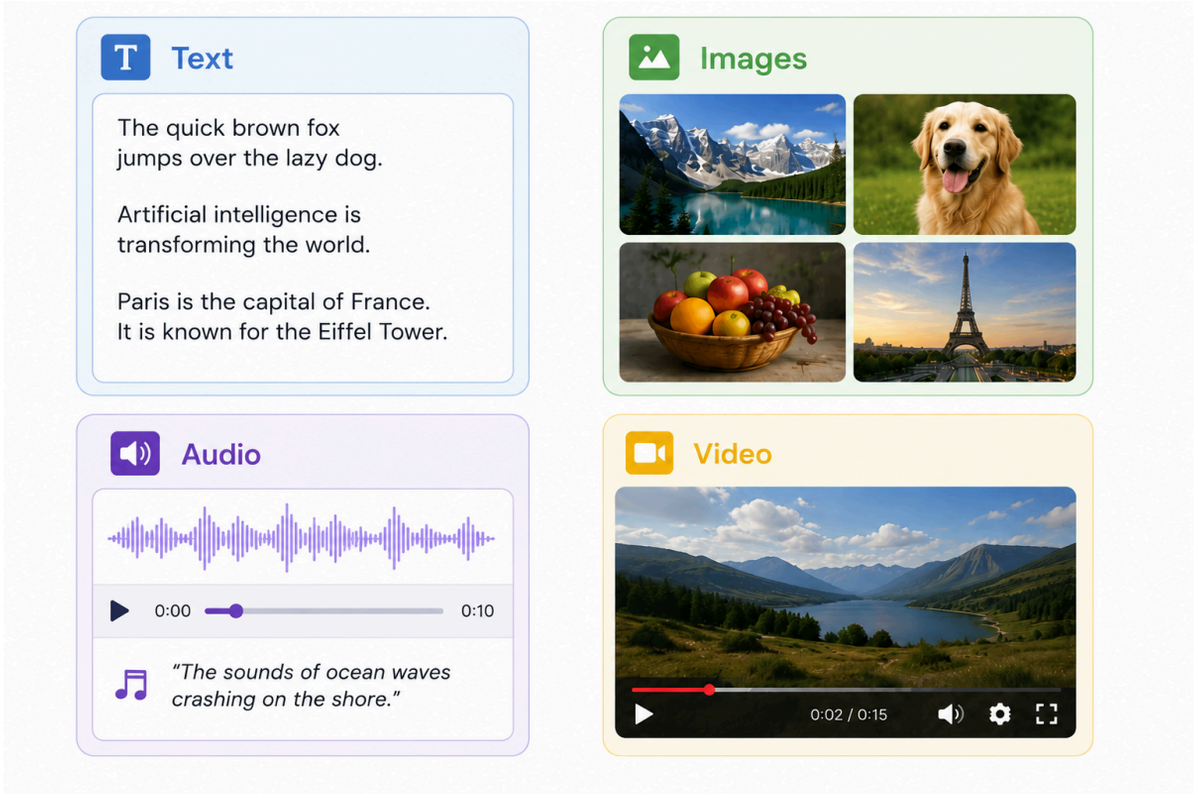
Lecture 17: multimodal models

So far: language models

text ⇒ text

The world is multimodal:



> 图：多模态示意：以图标列出多种模态——图像（image）、视频（video）、文本（text）、音频（audio），以及味觉（taste）、嗅觉（smell）、触觉（touch）。

> 第 17 讲：多模态模型 目前为止：语言模型 文本 ⇒ 文本 世界是多模态的：

Ultimate goal: **omni model**

- Input any combination of modalities (understanding)
- Output any combination of modalities (generation)

> 终极目标：omni 模型 输入任意模态组合（理解） 输出任意模态组合（生成）

Where we are today:

- Transformers work really well. So we gotta use them.
- Transformers speak tokens (discrete or continuous), where a token represents some ~semantic unit of information.
- Therefore, we must convert everything into tokens.
- Note: we had to do this with text (recall the tokenization lecture).
- For non-text modalities, this is more challenging...

> 我们今天的现状： Transformer 效果非常好，所以我们得用它。 Transformer 以 token 为语言（离散或连续），每个 token 大致代表一个语义层面的信息单元。 因此，我们必须把一切都转换成 token。 注意：对文本我们也不得不这么做（回忆 tokenization 那一讲）。 对非文本模态来说，这更具挑战性……

Questions:

1. How do we input non-text data (e.g., understand images)?
2. How do we output non-text data (e.g., generate audio)?

> 问题： 如何输入非文本数据（例如理解图像）？ 如何输出非文本数据（例如生成音频）？

Summary:

- Frontier models are expected to be multimodal (natively multimodal, omni)
- Fundamental challenge: how to encode non-text modalities?
- Comprehension and generation might demand different things (semantics versus finer-grained details)
- Balance images + video (lower information density) and text for training stability

- Continuous encoders + Transformer + diffusion models for generation

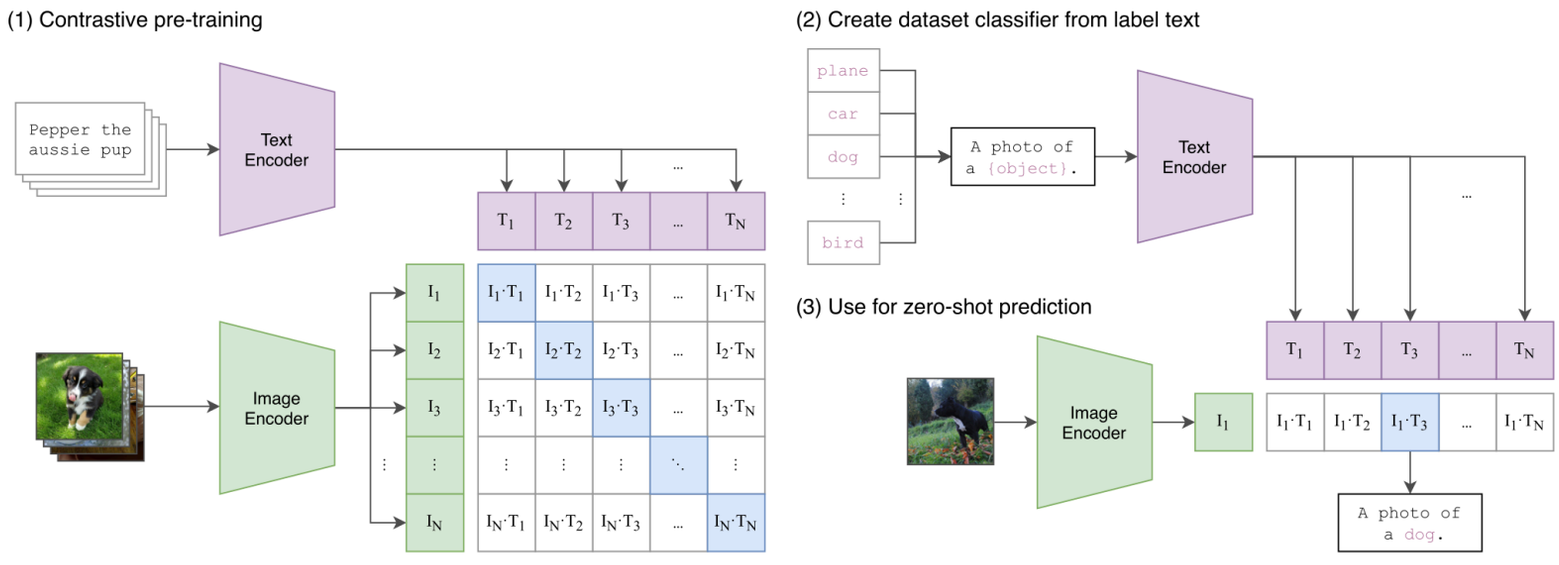
> 总结： 前沿模型被期望是多模态的（原生多模态、omni） 根本挑战：如何编码非文本模态？ 理解与生成可能需要不同的东西（语义 vs 更细粒度的细节） 为了训练稳定性，需要在图像 + 视频（信息密度较低）与文本之间取得平衡 连续编码器 + Transformer + 扩散模型负责生成

CLIP (Contrastive Language-Image Pretraining) [Radford+ 2021]

> CLIP（对比式语言-图像预训练） [Radford+ 2021]

Context:

- Computer vision models were trained on annotated images.
- Question: is it possible to leverage the much larger amount of (image, caption) pairs?



> 图：CLIP 对比学习示意——N 个图文对分别经过图像编码器（ResNet/ViT）与文本编码器（Transformer）得到特征 I1…IN 与 T1…TN，计算 N×N 余弦相似度矩阵，最大化对角线上的匹配对、压低非匹配对（对比目标函数）。

> 背景： 计算机视觉模型是在带标注的图像上训练的。 问题：能否利用规模大得多的（图像, 描述）对?

Method:

- Get a batch of (image, text) examples (e.g., 32768)
- Encode each image and each text
- For each image, prefer its aligned text over other texts
- For each text, prefer its aligned image over other images

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

> 图：CLIP 对比预训练伪代码：image_encoder 与 text_encoder 分别编码对齐的图像 I 与文本 T 小批量，经投影矩阵 W_i、W_t 得到联合嵌入，计算缩放的余弦相似度 logits，再对图→文与文→图两个方向取对称交叉熵损失。

> 方法： 取一批（图像, 文本）样本（例如 32768 个） 编码每张图像和每段文本 对每张图像，偏向其配对的文本而非其他文本 对每段文本，偏向其配对的图像而非其他图像

Data:

- Searched for 500K queries, get ~20K (image, text) pairs per query

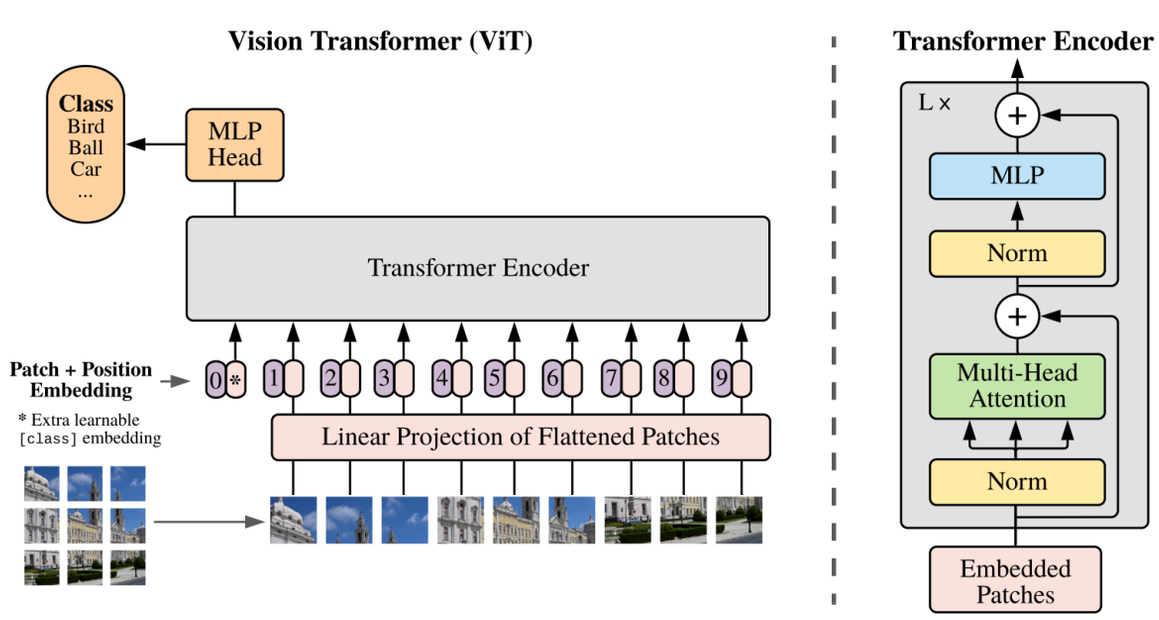
- Trained on 400M image-text pairs
- Didn't release the dataset
- Reproduced in OpenCLIP (using LAION-5B dataset, which used CLIP for filtering) [Cherti+ 2022]

> 数据： 搜索了 500K 个查询，每个查询得到约 20K 个（图像, 文本）对 在 400M 图文对上训练 没有公开数据集 在 OpenCLIP 中被复现（使用 LAION-5B 数据集，而该数据集本身用 CLIP 做了过滤） [Cherti+ 2022]

Data processing [code]
- Images come in all resolutions (arbitrary W x H)
- Resize using bicubic interpolation so shorter side is 336 pixels
- Center crop (cuts off borders to get 336 x 336)

> 数据处理 [code] 图像有各种分辨率（任意的 W x H） 用双三次插值调整大小，使较短边为 336 像素 中心裁剪（切掉边缘以得到 336 x 336）

Vision encoder:
- Experimented with ResNet-50 and Vision Transformers [Dosovitskiy+ 2020]



> 图：Vision Transformer（ViT）结构：图像切分为图块（patch）并展平，经线性投影（Linear Projection of Flattened Patches）并加位置嵌入（含额外可学习的 [class] 嵌入），送入 Transformer 编码器（LN、多头自注意力、MLP 模块），最后由 MLP Head 输出类别（如 Bird、Ball、Car 等）。

- Attention pooling: do QKV with query = global average of activations
- Best model: ViT-L/14@336px (L = large, 14x14 patches, 3 channels, trained on 336x336 resolution images)

> 视觉编码器： 实验了 ResNet-50 和 Vision Transformer [Dosovitskiy+ 2020] 注意力池化（attention pooling）： 以 query = 激活值的全局平均来做 QKV 最佳模型：ViT-L/14@336px（L = large，14x14 patch，3 通道，在 336x336 分辨率图像上训练）

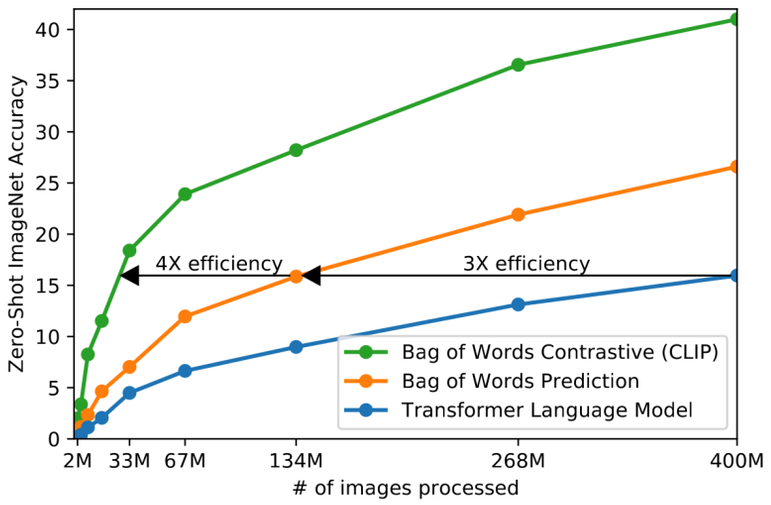
Text encoder:
- GPT-2 Transformer (63M parameters, 12 layers)
- Encode [BOS] ... [EOS], return [EOS] activation at highest layer

> 文本编码器： GPT-2 Transformer（63M 参数，12 层） 编码 [BOS] ... [EOS]，返回最高层的 [EOS] 激活值

Headline result:
- On ImageNet, zero-shot CLIP outperformed ResNet-50 trained on 1.2M ImageNet images

> 主要结果： 在 ImageNet 上，zero-shot CLIP 超过了在 1.2M 张 ImageNet 图像上训练的 ResNet-50

Ablation:
- Alternative: predict text from images directly
- Much less compute efficient compared to CLIP-style ranking



> 图：「CLIP 的高效预训练（Efficient Pretraining of CLIP）」双图：左图横轴为已见样本数（十亿）、纵轴为 ImageNet 零样本准确率，比较 SigLIP 2（webli）、CLIPA(@336)、SigLIP（webli）、CLIP（HT）、FLIP 等方法的样本效率；右图同样以样本数为横轴，从计算效率角度对比这些方法，展示高效预训练策略以更少样本达到更高准确率。

> 消融实验： 替代方案：直接从图像预测文本 与 CLIP 式排序相比，计算效率低得多

Summary:

- Encoding of images captures semantics given by (noisy) text
- Design decisions chosen based on image classification (not very fine-grained)
- Technical: requires large batch sizes, softmax operation over full batch

> 总结： 图像的编码捕捉到了（带噪声的）文本所给出的语义 设计决策是基于图像分类任务做出的（并不很细粒度） 技术层面：需要很大的 batch size，且 softmax 要在整个 batch 上做

SigLIP (Sigmoid Loss for Language Image Pre-Training) [Zhai+ 2023]

> SigLIP（用于语言-图像预训练的 Sigmoid 损失） [Zhai+ 2023]

Objective:

- CLIP: multiclass classification for (text, image) versus (text, image') for all image'
- SigLIP: binary classification for (text, image) - aligned or not?

Algorithm 1 Sigmoid loss pseudo-implementation.

```
# img_emb      : image model embedding [n, dim]
# txt_emb      : text model embedding [n, dim]
# t_prime, b   : learnable temperature and bias
# n            : mini-batch size

t = exp(t_prime)
zimg = l2_normalize(img_emb)
ztxt = l2_normalize(txt_emb)
logits = dot(zimg, ztxt.T) * t + b
labels = 2 * eye(n) - ones(n) # -1 with diagonal 1
l = -sum(log_sigmoid(labels * logits)) / n
```

> 图：SigLIP 损失代码片段——代码先计算图文特征点积 logits = x @ y.T，再用逐对的 sigmoid 损失（-log(sigmoid(τ·(logits·label))) 形式，τ 为可学习温度、label 取 ±1）替代 CLIP 的 softmax 对比损失。

> 目标函数： CLIP：将 (文本, 图像) 与所有 (文本, 图像') 做多分类 SigLIP：对 (文本, 图像) 做二分类——是否配对?

Data:

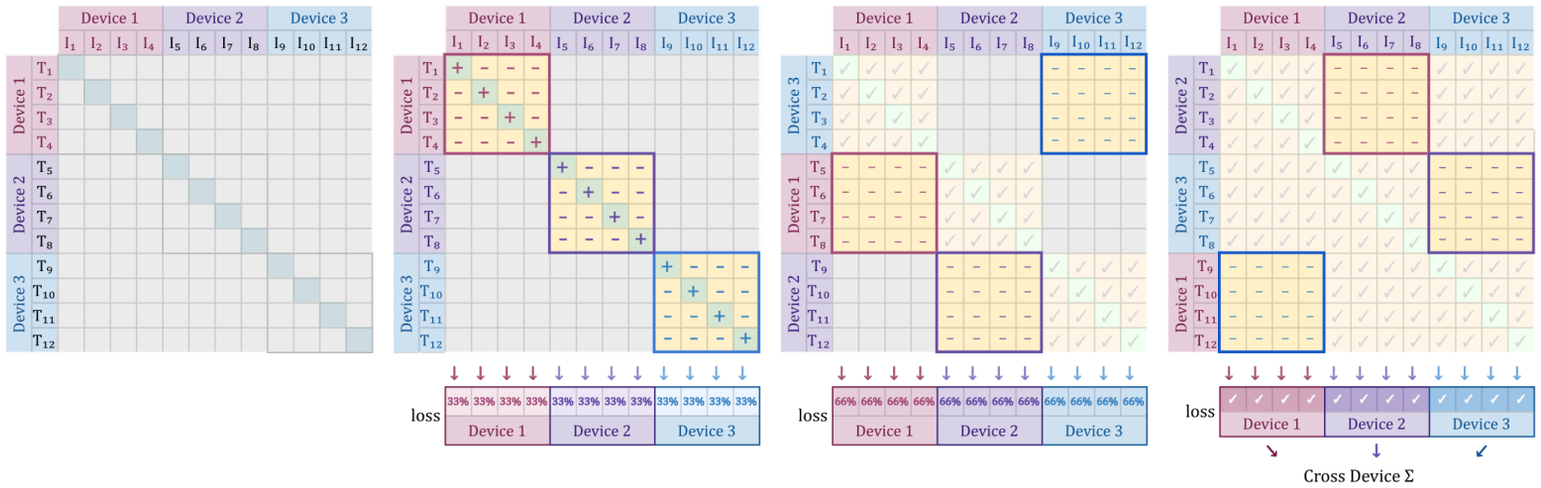
- WebLI dataset: O(billion) (image, text) pairs [Chen+ 2022]
- Scraped from the Internet
- Used automatic OCR to extract text from images
- Keep 10% highest quality
- Supports 100 languages

> 数据： WebLI 数据集：O(billion) 量级的（图像, 文本）对 [Chen+ 2022] 从互联网抓取 用自动 OCR 从图像中提取文本 保留质量最高的 10% 支持 100 种语言

Efficiency:

- CLIP: 10 days on 256 TPUv3

- SigLIP: 5 days on 32 TPUv4 (lower FLOP/s than TPUv3) - much faster!



> 图：SigLIP 训练并行策略示意（多个 panel）：Device 1/2/3 各持有文本批 T_1–T_{12} 与图像批 I_1–I_{12} 的不同划分，展示不同数据/模型并行方式下各设备上的布局。

> 效率： CLIP：在 256 个 TPUv3 上 10 天 SigLIP：在 32 个 TPUv4 上 5 天（FLOP/s 低于 TPUv3）——快得多！

Batch size:

- Decouple batch size from loss
- Better than CLIP for <16K batch sizes
- Go up to 1M batch size, but 32K is enough

> Batch size： 将 batch size 与损失解耦 在 <16K 的 batch size 下优于 CLIP 可以上到 1M 的 batch size，但 32K 就够了

LLaVA (Large Language and Vision Assistant) [Liu+ 2023]

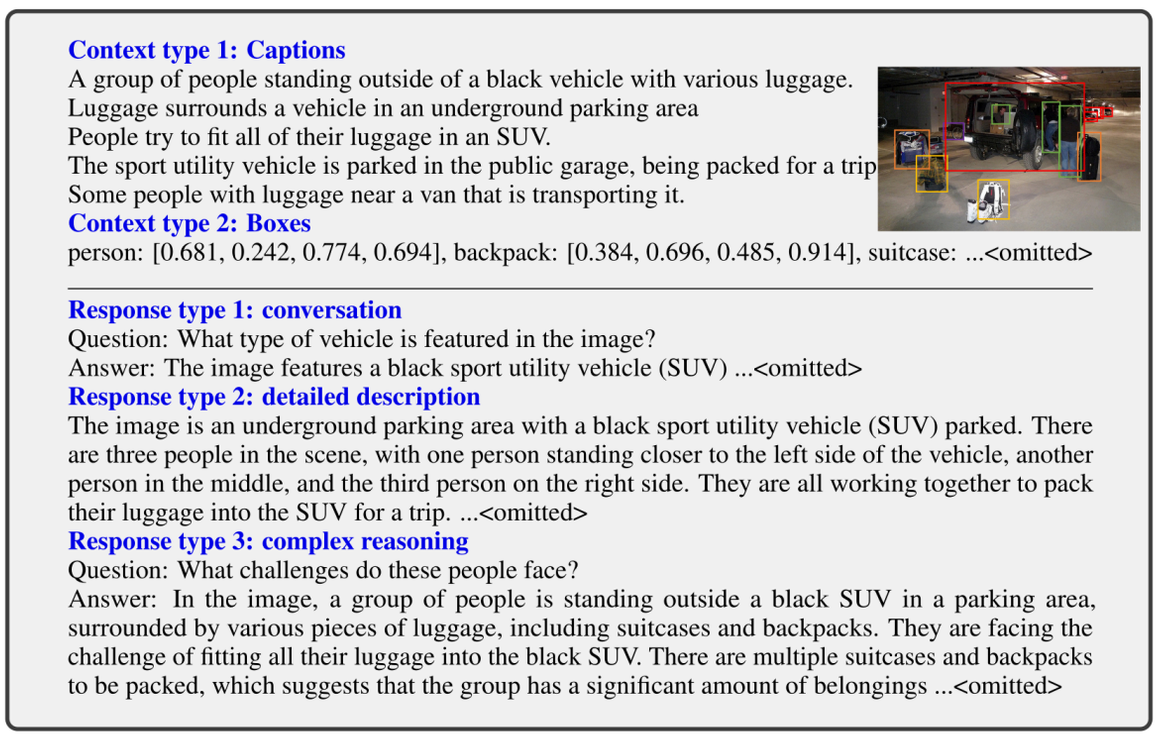
> LLaVA（大型语言与视觉助手） [Liu+ 2023]

Vision encoder: CLIP
Text decoder: Vicuna (LLaMA fine-tuned on ShareGPT conversations) [post]

> 视觉编码器：CLIP 文本解码器：Vicuna（在 ShareGPT 对话上微调的 LLaMA） [post]

Data:

- MS COCO has images annotated with bounding boxes and Mechanical Turk captions
- Prompt GPT-4 with captions or detected objects and generate questions or conversations
- Pair generations with original images
- 158K examples

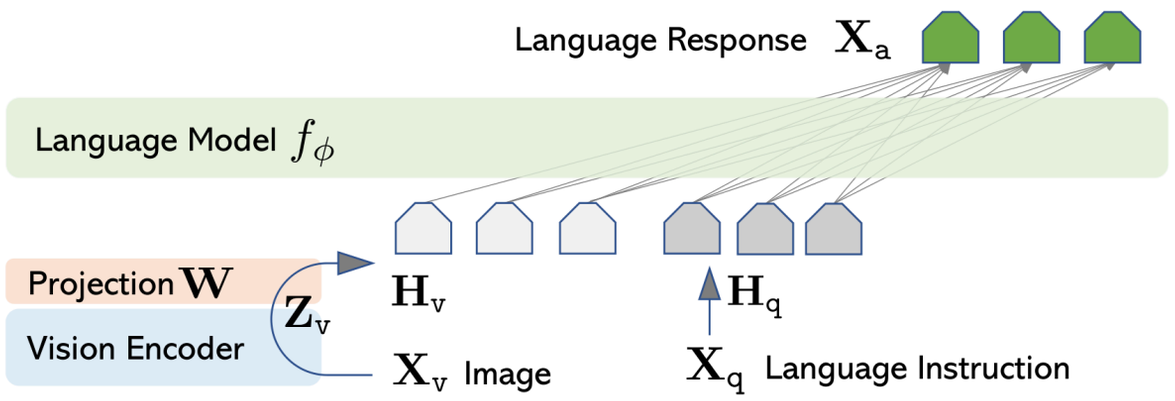


> 图：LLaVA-Gen 用 GPT 生成的三类视觉指令数据：① 图片描述（captions），如“一群人带着行李站在黑色车辆外”；② 单轮响应（responses）；③ 多轮对话（conversations），均附有示例文本。

> 数据： MS COCO 的图像带有边界框标注和 Mechanical Turk 撰写的描述 用图像描述或检测到的物体去 prompt GPT-4，生成问题或对话 把生成结果与原始图像配对 158K 个样本

Model:

- Encode images with CLIP (ViT-L/14)
- Linear projection (W) into embedding space (Flamingo and Q-former are more complex)



> 图：LLaVA 架构：阶段 1「面向特征对齐的预训练」——用 CLIP ViT-L/14 编码图像得到图像 token 的隐状态 X_v，经线性投影（Linear Projection）映射到语言模型 Vicuna 的输入空间，此阶段只训练投影层；阶段 2「面向指令跟随的微调」——用多模态指令跟随数据微调（含 FFN），使模型按指令回答关于图像的问题。

> 模型： 用 CLIP（ViT-L/14）编码图像 线性投影（W）映射到嵌入空间（Flamingo 和 Q-former 更复杂）

Training:

- Stage 1 (alignment): freeze vision encoder and language model, only train W
- Stage 2 (fine-tuning): freeze vision encoder and train W and language model

Visual input example, Extreme Ironing:



Source: https://www.barnorama.com/wp-content/uploads/2016/12/03-Confusing-Pictures.jpg

User	What is unusual about this image?
LLaVA	The unusual aspect of this image is a man ironing clothes on the back of a minivan or van. This is not a typical place to perform this activity, as one would usually iron clothes in a more stationary and safe location, such as a home, using a regular ironing board. The scene depicted in the image is peculiar as it involves a makeshift ironing setup on a vehicle, which can be both unsafe and unconventional. Additionally, it is not clear how the man is able to maintain balance and stability while ironing clothes in such an unstable environment. [*Start a new conversation, and clear the history*]
User	What's happening in the scene?
LLaVA	The image depicts a man standing on top of a yellow SUV in a busy city street. He is holding a portable ladder, seemingly ironing clothes while standing on the vehicle. Around the scene, there are other cars, a traffic light, and a fire hydrant. Additionally, there are two other people in the vicinity, one close to the man on the car and another further away. The man on the ladder appears to be attracting attention due to his unconventional choice of ironing his clothes on top of a moving car. The city street around him is bustling with activity, adding to the unique nature of the scene.
User	What is unusual about this image?
GPT-4 [36]	The unusual thing about this image is that a man is ironing clothes on an ironing board attached to the roof of a moving taxi.
User	What is unusual about this image?
BLIP-2	a man is sitting on the back of a yellow cab
User	What is unusual about this image?
OpenFlamingo	The man is drying his clothes on the hood of his car.

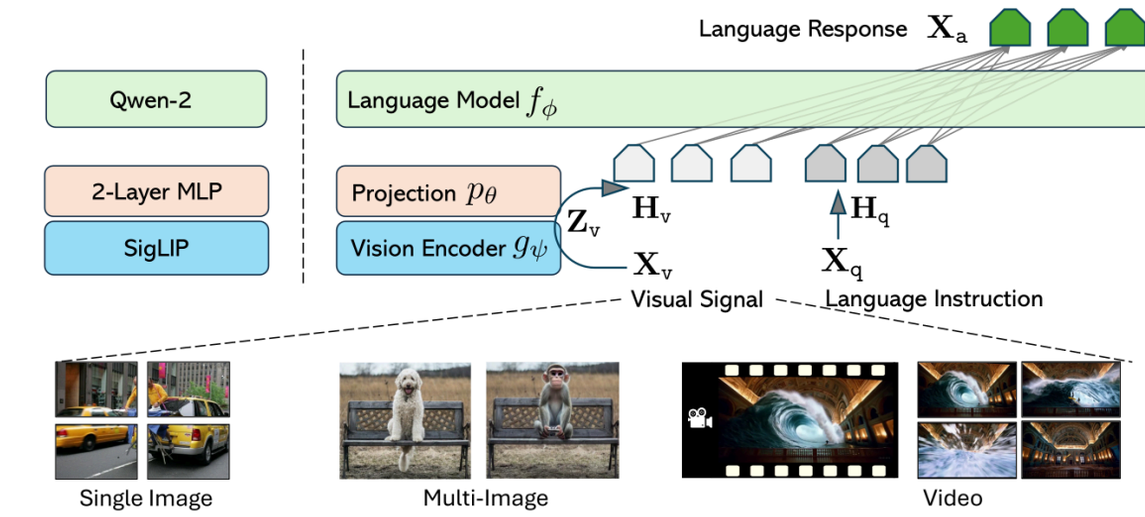
> 图：LLaVA 对话示例——左侧输入厨房场景照片，模型回答“此人正在揉面准备食物”之类的描述；右侧输入图表类图片，模型解读其内容，展示多模态对话理解能力。

> 训练： 阶段 1（对齐）：冻结视觉编码器和语言模型，只训练 W 阶段 2（微调）：冻结视觉编码器，训练 W 和语言模型

LLaVA OneVision [Li+ 2024]

- Latest version in the LLaVA series (after LLaVA 1.5, LLaVA-Next)
- Handle multiple images, video

> LLaVA OneVision [Li+ 2024] LLaVA 系列的最新版本（在 LLaVA 1.5、LLaVA-Next 之后） 处理多图像、视频



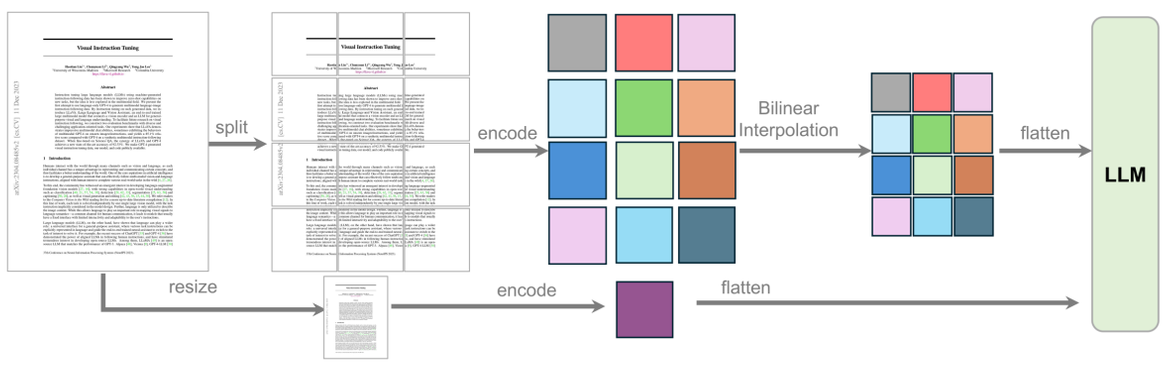
图：LLaVA-OneVision 架构图：图像/视频 token 经 2 层 MLP 投影（AnyRes）后，与文本 token 一同输入语言模型（Qwen-2，Language Model f_ϕ）生成回答。

- Vision encoder: SigLIP (use grid features before and after last Transformer layer)
- Text decoder: Qwen-2 72B
- Projector: 2-layer MLP

视觉编码器：SigLIP（使用最后一个 Transformer 层前后的网格特征） 文本解码器：Qwen-2 72B 投影器：2 层 MLP

Data processing:

- Preserving high resolution is important (e.g., for OCR)
- CLIP resizes and crops to 336x336, which loses information
- Solution: AnyRes, introduced in LLaVA 1.5 [paper]
- Break up image into a x b pieces (matching resolution of vision encoder), encode, concatenate
- If too many tokens (original image is too high resolution), then use bilinear interpolation



图：LLaVA-OneVision 的 AnyRes 编码：高分辨率图像按网格切分成多个子块（如 2×2、2×3、3×2），各块分别经 ViT 编码、通过双线性插值（bilinear interpolation）对齐位置嵌入后展平；同时整图缩放到基准分辨率单独编码，两条路径的视觉 token 一并送入 LLM（如 336×336 基准可扩至 672×672、最多 24 块共 576 个视觉 token）。

Handle 3 types of input (single image, multiple images, video):

- Goal: make all of the modalities produce roughly the same length



图：LLaVA-OneVision 的表示与模态设计：视觉表示（Visual representations）与语言表示（Language representations，仅语言 Language only）相对，另有三个扩展方向——从语言到视觉（联合）、从单图到多图、从图像到场景文本再到视频；对应五个训练面板：Language、单图（Single image）、多图（Multi-image）、场景文本（Scene text）、视频（Video）。

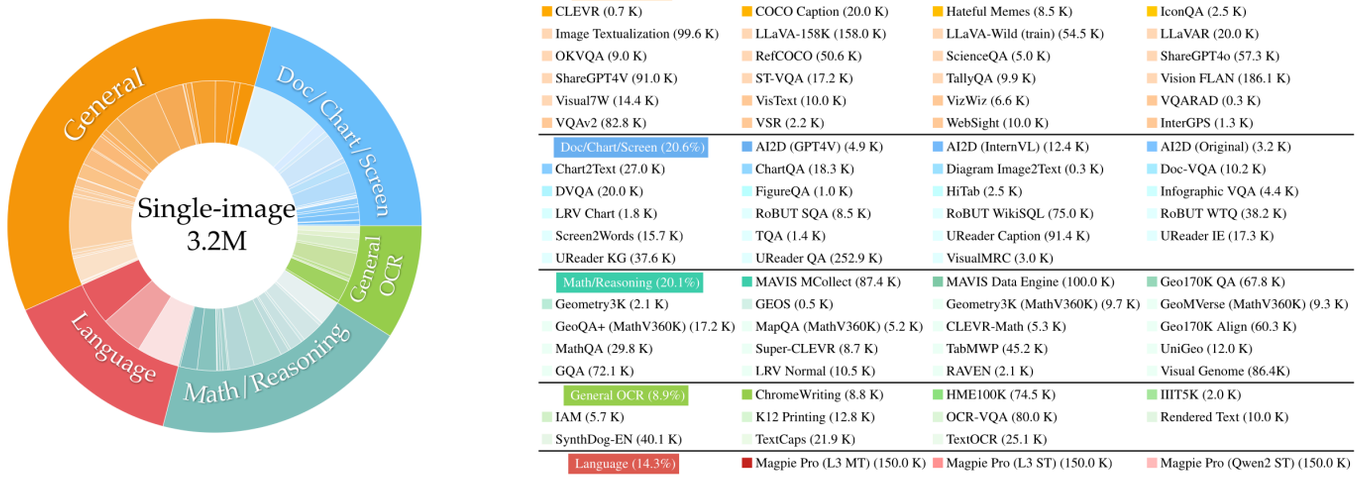
- Single image: use higher resolution
- Multiple images: use base resolution for each image

- Video: use lower resolution for each frame

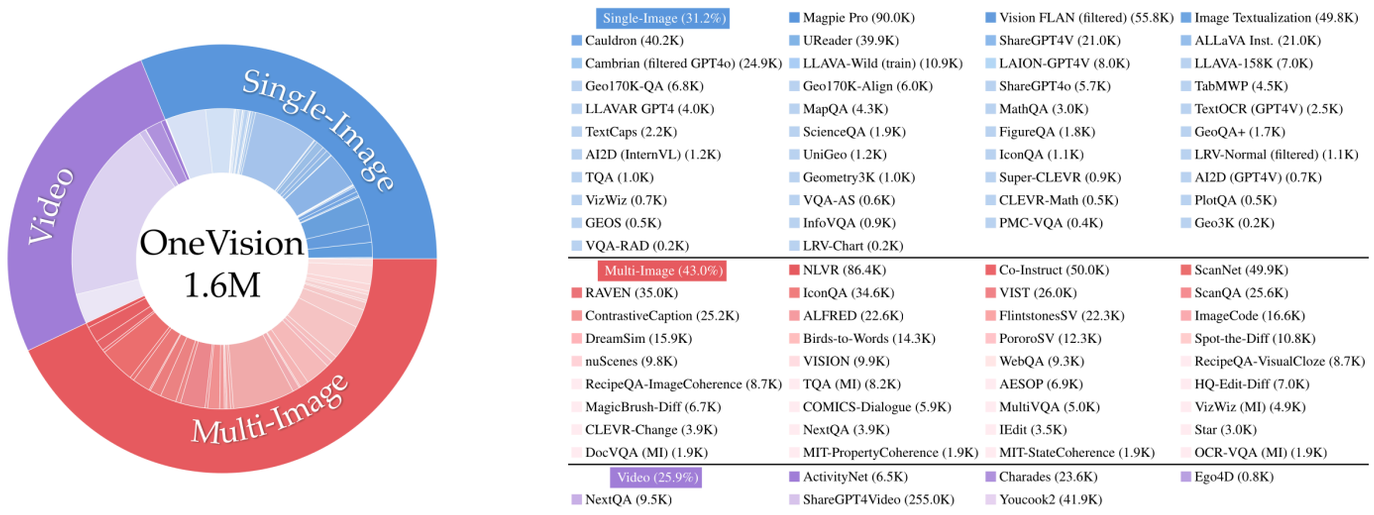
> 数据处理： 保留高分辨率很重要（例如对 OCR） CLIP 会缩放并裁剪到 336x336，这会损失信息 解决方案：AnyRes，在 LLaVA 1.5 中引入 [paper] 把图像拆成 a x b 块（与视觉编码器的分辨率匹配），逐块编码，再拼接 如果 token 数太多（原图分辨率太高），就用双线性插值 处理 3 类输入（单图像、多图像、视频）： 目标：让所有模态产出的长度大致相同 单图像：用更高分辨率 多图像：每张图用基础分辨率 视频：每一帧用更低分辨率

Data:

- Philosophy: quality over quantity



> 图：LLaVA-OneVision 三阶段训练数据构成——各数据源在阶段 1（语言-图像概念对齐）、阶段 2（高质量多模态指令微调）、阶段 3（单图扩展到多图/视频）中的使用情况，以条形图展示各阶段的数据混合。

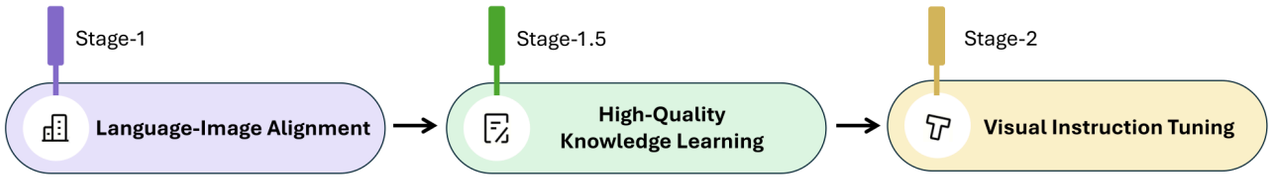


> 图：OneVision 160 万样本构成环形图：Single-Image 31.2%、Multi-Image 43.0%、Video 25.9%，并详列 Magpie Pro、Vision-FLAN、ShareGPT4Video 等来源数据集及规模。

> 数据： 理念：质量重于数量

Training:

- Philosophy: easier to harder



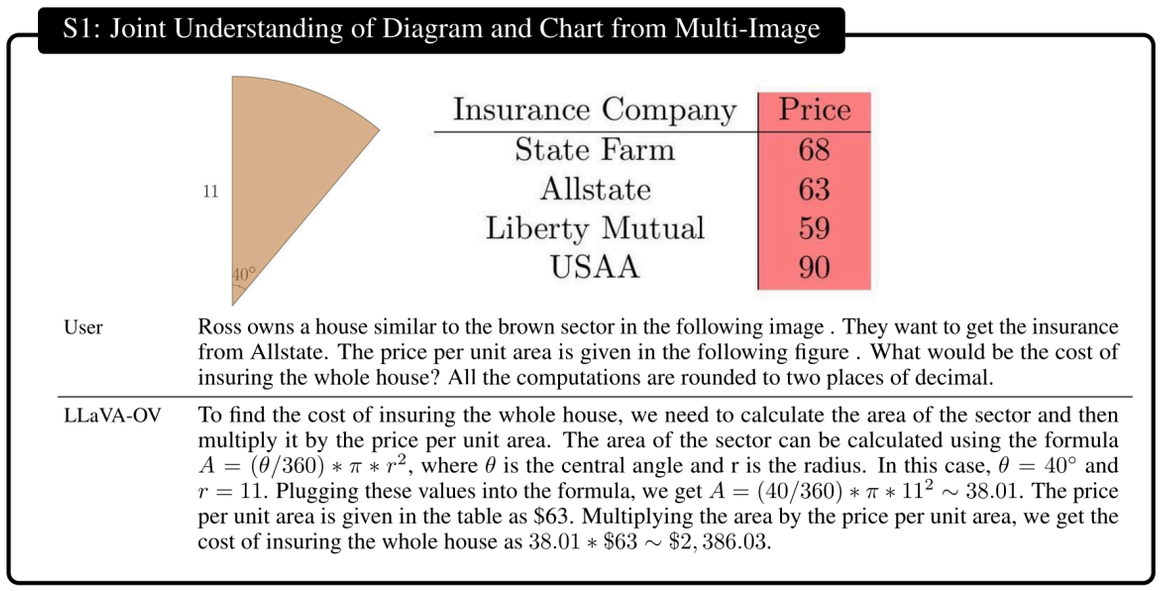
		Stage-1	Stage-1.5	Stage-2	
				Single-Image	OneVision
Vision	**Resolution**	384	384×{2×2, 1×{2,3}, {2,3}×1}	384×{{1×1}, ⋯, {6×6}}	384×{{1×1}, ⋯, {6×6}}
	#Tokens	729	Max 729×5	Max 729×10	Max 729×10 (See Fig. 3)
Data	**Dataset**	LCS	Image (Sec. 4.1)	Image (Sec. 4.2)	(Multi)-Image & Video (Sec. 4.2)
	#Samples	558K	4M	3.2M	1.6M
Model	**Trainable**	Projector	Full Model	Full Model	Full Model
	0.5B LLM	1.8M	0.8B	0.8B	0.8B
	7.6B LLM	20.0M	8.0B	8.0B	8.0B
	72.7B LLM	72.0M	73.2B	73.2B	73.2B
Training	**Batch Size**	512	256/512	256/512	256/512
	LR: ψ_{vision}	1×10^{-3}	2×10^{-6}	2×10^{-6}	2×10^{-6}
	LR: $\{\theta_{\text{proj}}, \phi_{\text{LLM}}\}$	1×10^{-3}	1×10^{-5}	1×10^{-5}	1×10^{-5}
	Epoch	1	1	1	1

图：LLaVA-OneVision 的三阶段训练：Stage-1 语言-图像对齐、Stage-1.5 高质量知识学习、Stage-2 视觉指令微调；上行为单图像（Single-Image）路径，下行为 OneVision 路径（多图/视频），各阶段使用不同输入分辨率（384 及 2×2、1×2、1×3 等切块组合，最高 6×6）。

训练： 理念：由易到难

Transfer between modalities:

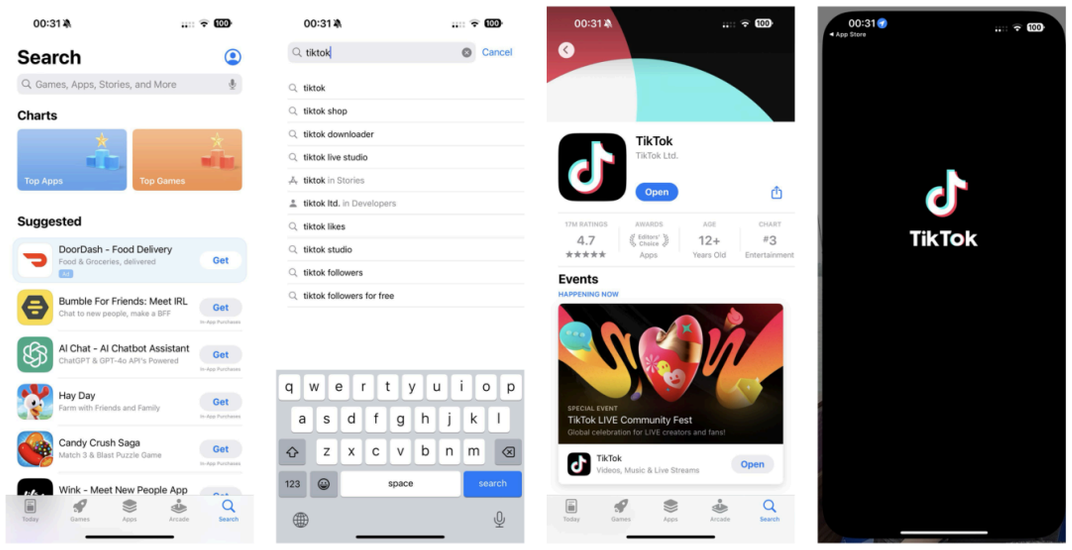
- Single image data for diagrams and charts, but generalize to multi-image



图：「单阶段预训练后的迁移性能（Transfer performance after single-stage pretraining，OV-1.0-7B）」：横轴为预训练轮数（0–40），纵轴为平均性能（7 个基准平均，约 0.50–0.80）；曲线比较单图（12 个基准）与视频（12 个基准）等设置下的迁移表现。

- OCR on single image data, relational reasoning from multi-image data, generalize to GUI-based agents

S2: Multi-image GUI Understanding for Agents

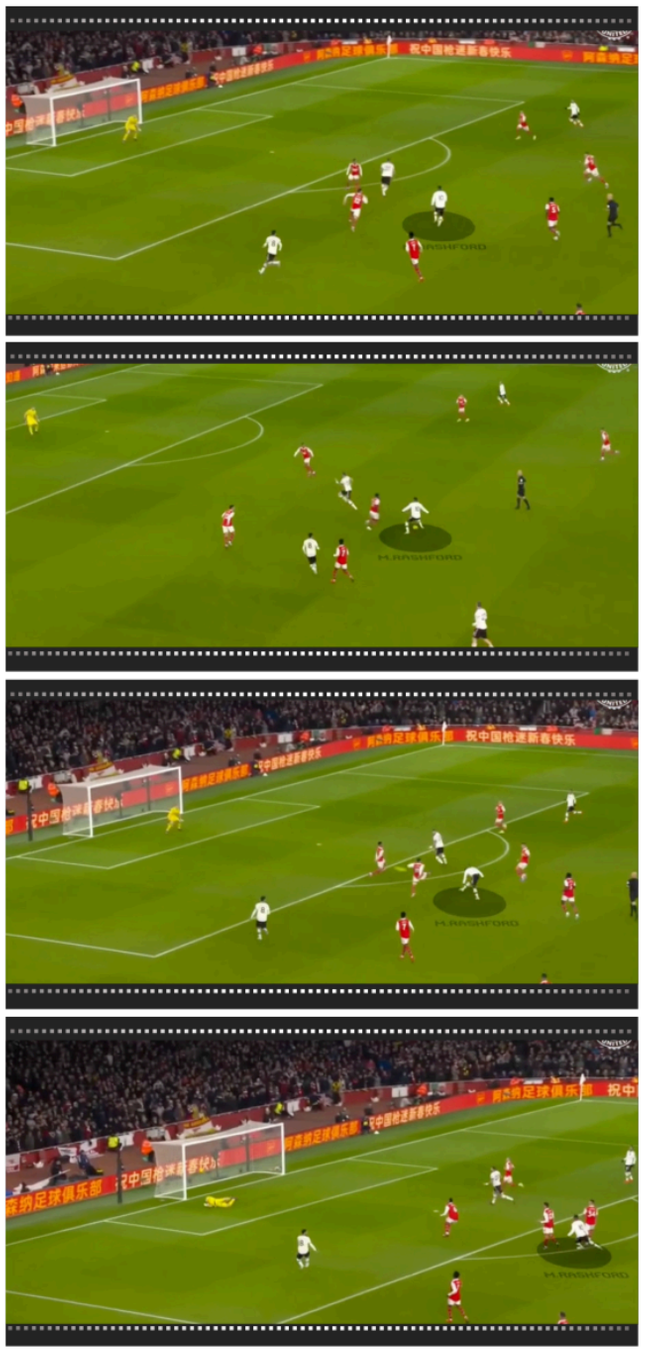


User	There are four sequential mobile UI screens, and are transformed by three steps of tap operations. Please respectively describe the three tap operations.
LLaVA-OV	The image shows a sequence of four mobile application interface screens. Here's a breakdown of the three tap operations based on the visible changes: 1. First Tap Operation: - The user starts by tapping on the search bar at the top of the screen. - The keyboard is shown at the bottom of the screen, indicating that the user can now type a search query. - The user types "TikTok" in the search bar and taps the search icon on the keyboard to initiate a search. 2. Second Tap Operation: - After tapping the search icon or pressing 'Enter', the screen displays a list of search results. - The user taps on the first result, which is the TikTok application. - This action opens the TikTok's app store page, which includes the app icon, developer information ... 3. Third Tap Operation: - The user taps the "Open" button on the app store page. - This action opens the TikTok application, and the screen transitions to the app's main interface.

> 图：LLaVA-OneVision 能力迁移柱状图——在多个基准（单图、多图、视频、通用问答等）上比较 LLaVA-1.5 与 LLaVA-OneVision 的性能，柱高表示相对提升幅度。

- Visual prompting (circle) in single images, generalize to videos

S8: Visual Prompting in Video



User	Describe the player highlighted in the video.
LLaVA-OV	The player highlighted in the video is wearing a white kit, and the number "10" is visible on their shirt. This player is positioned in the center of the pitch and is actively involved in the play, demonstrating control and dribbling skills as they navigate around the field. The player is surrounded by opponents in red kits, indicating that they are likely a key player for their team, possibly a playmaker or attacking midfielder. Throughout the video, the player maintains possession of the ball and creates opportunities for their team, showcasing their importance in the game's dynamics.

> 图：LLaVA-OneVision 示例 S8“Visual Prompting in Video”：用户要求“Describe the player highlighted in the video（描述视频中高亮的球员）”，模型给出其号码与表现。

> 模态间的迁移： 示意图与图表任务用单图像数据，但能泛化到多图像 OCR 用单图像数据，关系推理用多图像数据，泛化到基于 GUI 的 agent 在单图像中做视觉提示（画圈），泛化到视频

Summary:

- Standard VLM template: vision encoder + projector + LM
- Most work goes into data curation (heavy on synthesized, task-specific data)
- Open-source (released model weights and data)

> 总结： 标准 VLM 模板：视觉编码器 + 投影器 + LM 大部分功夫花在数据整理上（大量使用合成的、面向特定任务的数据） 开源（发布了模型权重和数据）

Qwen-VL [Bai+ 2023]

> Qwen-VL [Bai+ 2023]

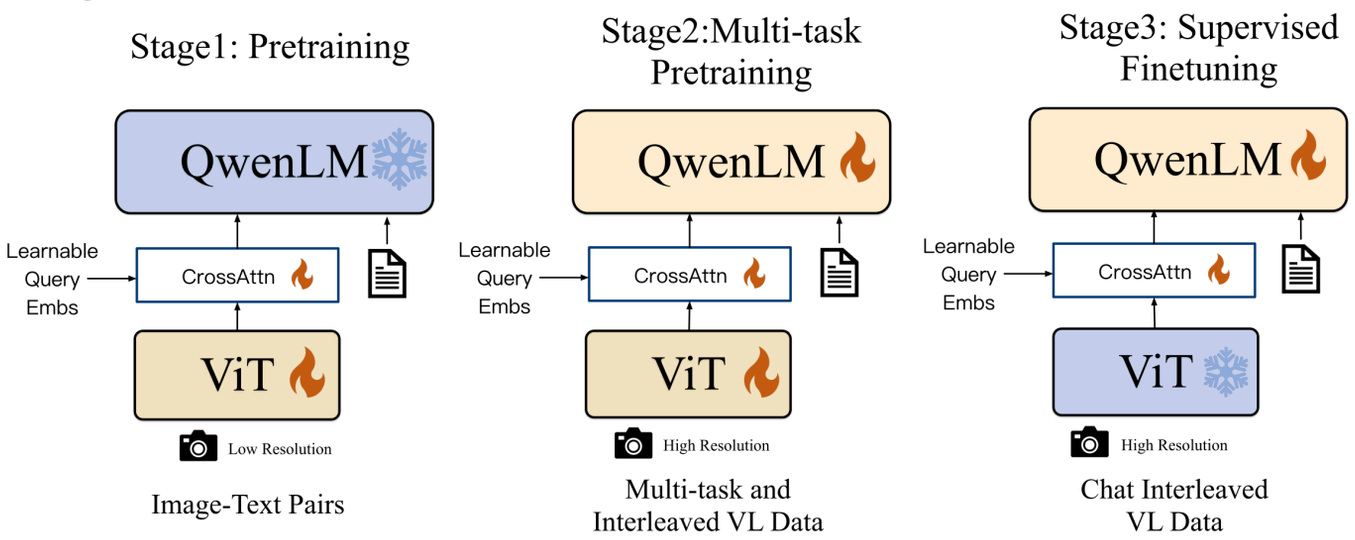
Architecture:

- Vision encoder: OpenCLIP's ViT-bigC (14x14 patches) [Cherti+ 2022]
- Adaptor: one layer cross-attention, incorporate 2D positional encodings, maps to fixed length of 256

- Special tokens: , ,

> 架构： 视觉编码器：OpenCLIP 的 ViT-bigC（14x14 patch） [Cherti+ 2022] 适配器：一层 cross-attention，融入 2D 位置编码，映射到固定长度 256 特殊 token：, ,

Training:



> 图：Qwen-VL 三阶段训练：Stage 1 预训练——图像-文本对、低分辨率，ViT 编码后经可学习查询嵌入（Learnable Query Embs）与交叉注意力（CrossAttn）接入 QwenLM；Stage 2 多任务预训练——高分辨率、多任务与交错图文数据；Stage 3 监督微调。

- Stage 1: large-scale low quality data; freeze LM, train vision encoder + adaptor

Language	Dataset	Original	Cleaned	Remaining%
English	LAION-en	2B	280M	14%
	LAION-COCO	600M	300M	50%
	DataComp	1.4B	300M	21%
	Coyo	700M	200M	28%
	CC12M	12M	8M	66%
	CC3M	3M	3M	100%
	SBU	1M	0.8M	80%
	COCO Caption	0.6M	0.6M	100%
Chinese	LAION-zh	108M	105M	97%
	In-house Data	220M	220M	100%
	Total	5B	1.4B	28%

> 图：Qwen-VL 第一阶段结构：图像与文本输入后，由冻结的 ViT（Qwen-VL）编码器把图像编码为 visual tokens，经一个带交叉注意力（cross-attention）的单层融合模块与语言特征融合，再送入 LLM（Qwen-7B）产生输出；此阶段仅训练融合模块（Trainable modules），ViT 与 LLM 保持冻结（Frozen）。

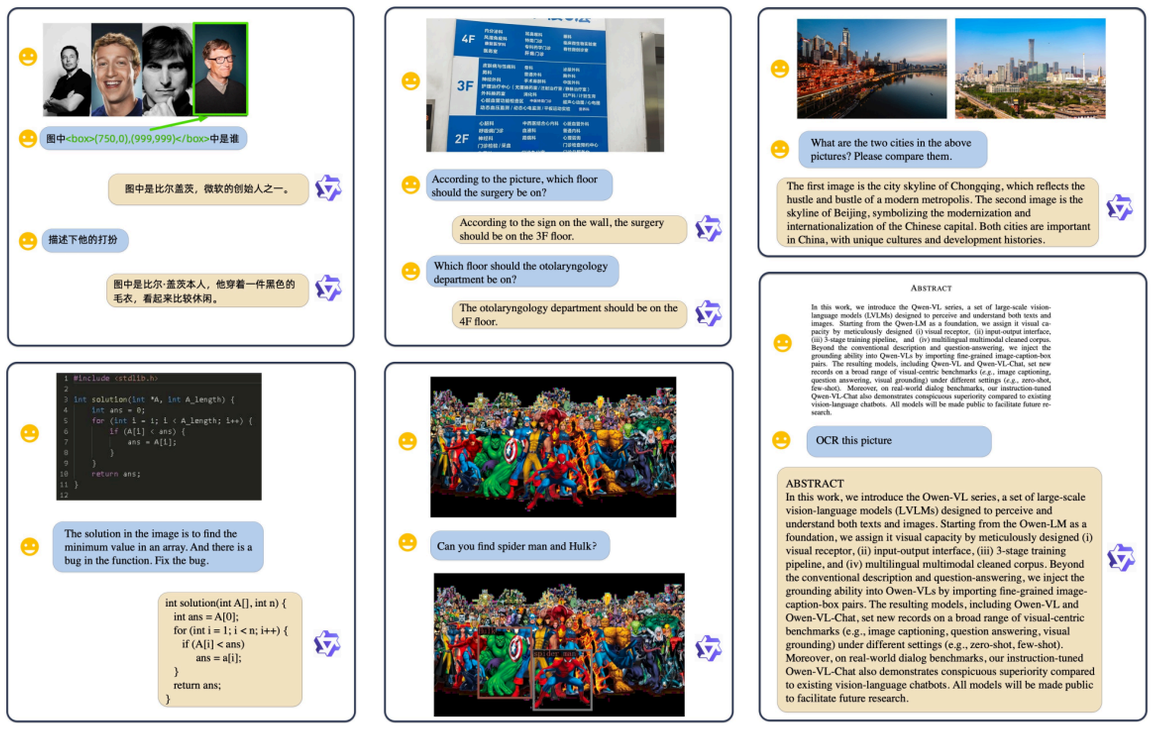
- Stage 2: higher quality task-specific data, increase resolution; train all parameters

Task	# Samples	Dataset
Captioning	19.7M	LAION-en & zh, DataComp, Coyo, CC12M & 3M, SBU, COCO, In-house Data
VQA	3.6M	GQA, VGQA, VQAv2, DVQA, OCR-VQA, DocVQA, TextVQA, ChartQA, AI2D
Grounding[2]	3.5M	GRIT
Ref Grounding	8.7M	GRIT, Visual Genome, RefCOCO, RefCOCO+, RefCOCOg
Grounded Cap.	8.7M	GRIT, Visual Genome, RefCOCO, RefCOCO+, RefCOCOg
OCR	24.8M	SynthDoG-en & zh, Common Crawl pdf & HTML
Pure-text Autoregression	7.8M	In-house Data

> 图：Qwen-VL 第二阶段（多任务预训练）示意——冻结 ViT 视觉编码器，使用更高分辨率的图文对（约 798M 图文数据、约 1.7B 参数规模）继续训练，同时引入多任务指令数据，各阶段数据规模标注在图中。

- Stage 3: instruction tuning data; freeze visual encoder, train adaptor + LM

> 训练： 阶段 1：大规模低质量数据；冻结 LM，训练视觉编码器 + 适配器 阶段 2：更高质量的任务特定数据，提高分辨率；训练全部参数 阶段 3：指令微调数据；冻结视觉编码器，训练适配器 + LM

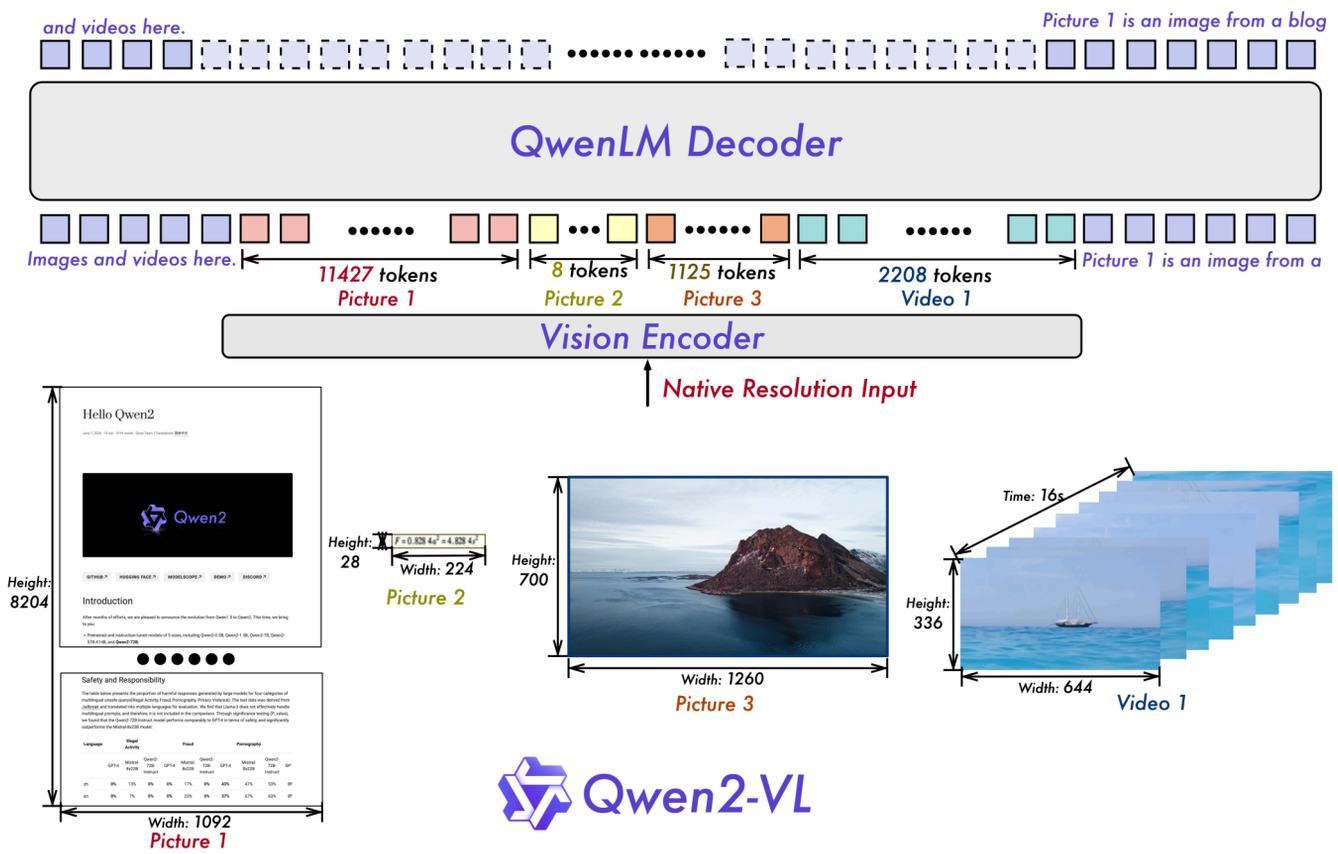


图：Qwen-VL 多任务示例面板：涵盖人物识别、地标、OCR、表格/文档理解等；提示多为中文，并用 <box>(x1,y1),(x2,y2)</box> 坐标框指定图中区域。

Qwen2-VL [Wang+ 2024]

> Qwen2-VL [Wang+ 2024]

Visual encoder: larger ViT (675M)



图：Qwen2-VL 架构：视觉编码器（Vision Encoder）支持原生分辨率输入，图像/视频被切成视觉 token 序列（如 Picture 1 仅 8 个 token，大图可达 1125/2208 个 token），与文本 token（“Hello Qwen2” 等）交替拼接，一同送入 QwenLM 解码器（LLM）处理，并采用 M-RoPE 多模态旋转位置编码。

- Key: dynamic resolution to handle varying resolutions
- Each 224 x 224 patch encoded with ViT/14, compress every 2x2 => 66 tokens
- Video: sample 2 frames/sec, max 16384 tokens

> 视觉编码器：更大的 ViT（675M） 关键：用动态分辨率应对各种不同的分辨率 每个 224 x 224 patch 用 ViT/14 编码，每 2x2 压缩 => 66 个 token 视频：每秒采样 2 帧，最多 16384 个 token

Multimodal Rotary Position Embedding (MRoPE):



> 图：Qwen2-VL 的 M-RoPE：标准 RoPE（1D）直接把序列位置编号 [1,2,...,12] 用于旋转位置编码；M-RoPE（2D，用于图像）把每个位置拆为时间 t、高度 h、宽度 w 三个分量分别编号（如 [1,1,1 | 2,2,2 | 3,3,3] 与 [1,2,3 | 1,2,3 | 1,2,3]）；用于视频时再扩展为三维（t、h、w），使位置编码与二维/时空网格结构对应。

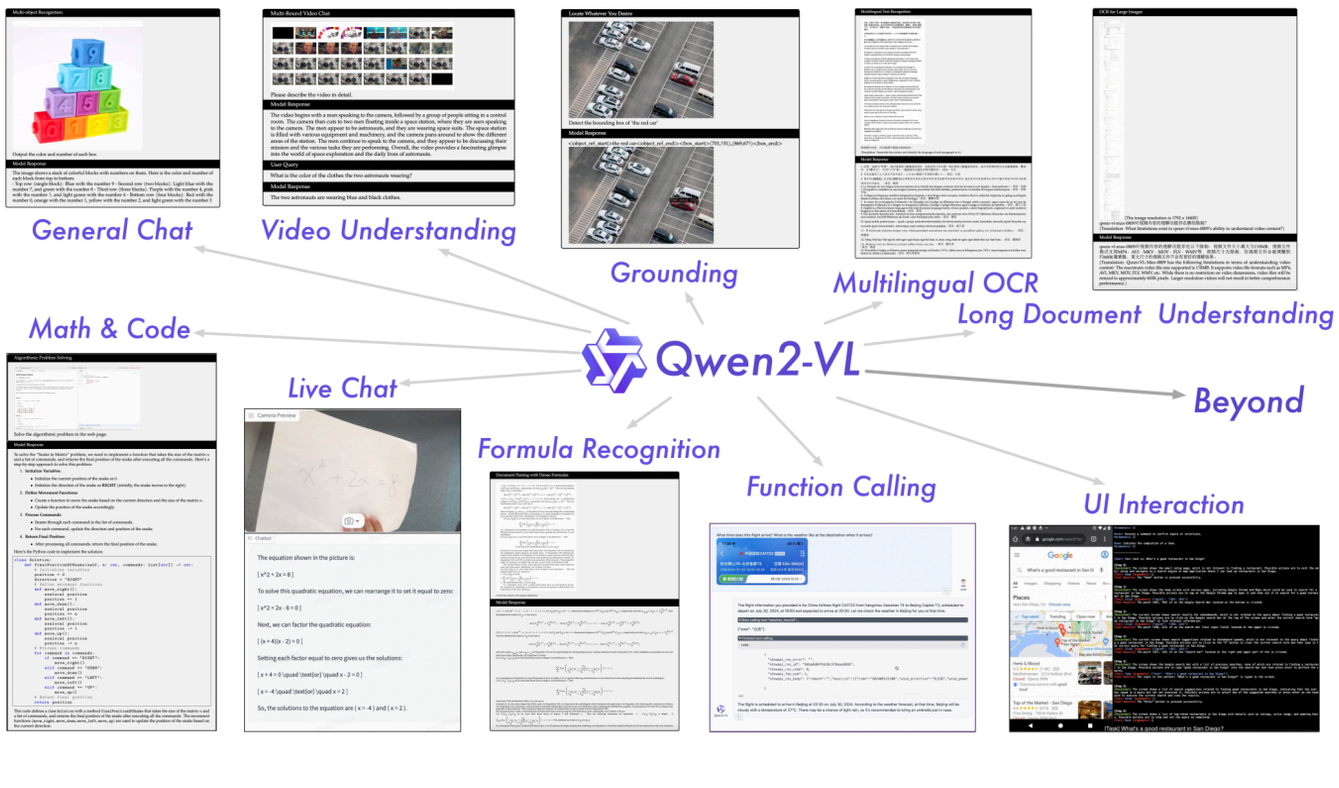
> 多模态旋转位置编码（MRoPE）：

Initialize LM with Qwen2 and vision encoder from DFN [Fang+ 2023]
Training (similar to Qwen-VL):

- Stage 1: train only visual encoder
- Stage 2: train all parameters
- Stage 3: train language model on instruction following datasets

> LM 用 Qwen2 初始化，视觉编码器来自 DFN [Fang+ 2023] 训练（与 Qwen-VL 类似）： 阶段 1：只训练视觉编码器 阶段 2：训练全部参数 阶段 3：在指令遵循数据集上训练语言模型

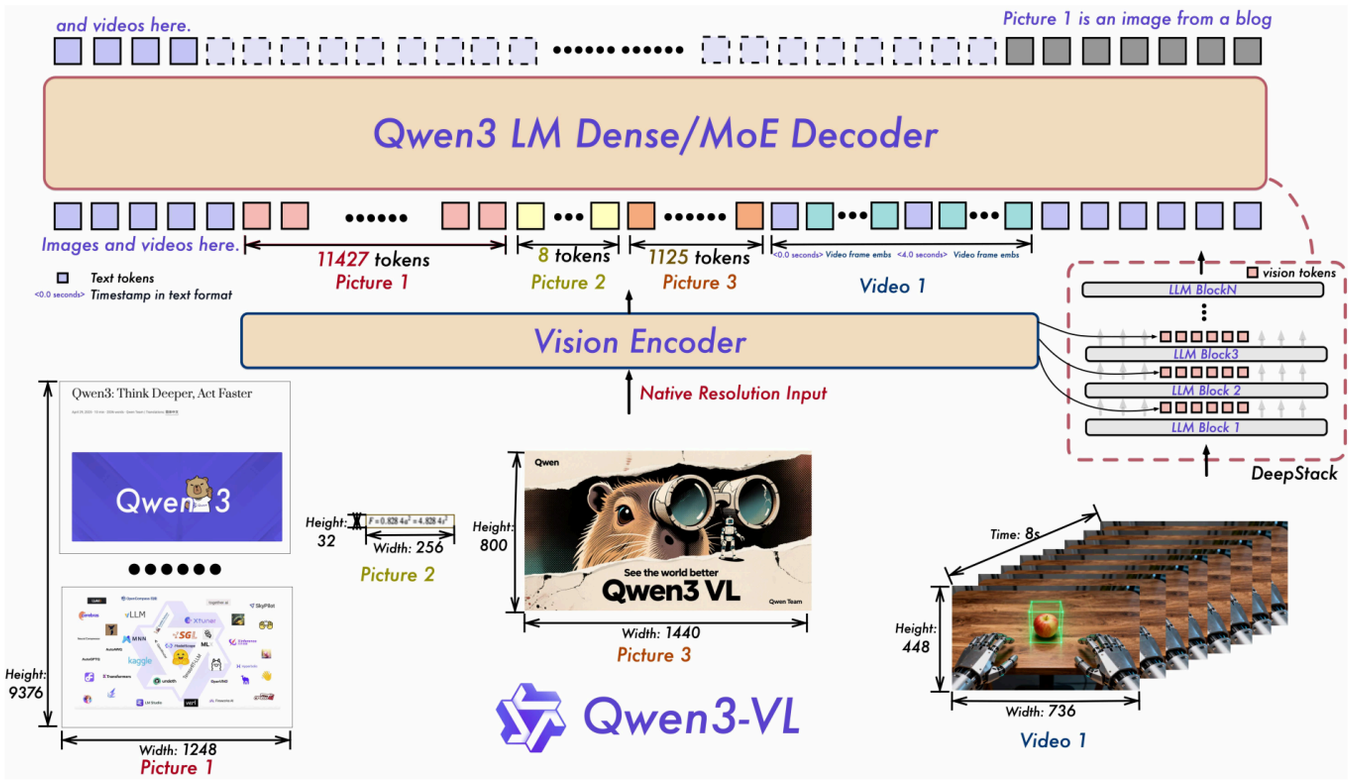
Many capabilities:



> 图：Qwen2-VL 能力展示——包括任意/动态分辨率图像理解、长视频理解与实时视频对话、多语言 OCR（如阿拉伯语文本识别）、视觉智能体操作界面等多个示例。

> 许多能力：

Qwen3-VL [Bai+ 2025]



图：Qwen3-VL 架构示意：图像与视频 token（如“Picture 1 is an image from a blog”）插入文本序列，由 Qwen3 LM（Dense/MoE Decoder）统一处理，实现原生多模态。

> Qwen3-VL [Bai+ 2025]

Language model:

- Qwen-3 models (dense and MoE models up to 235B-A22B)
- Long context understanding (256K)

> 语言模型： Qwen-3 模型（dense 与 MoE 模型，最大到 235B-A22B） 长上下文理解（256K）

Vision encoder:

- SigLIP-2 (same architecture as SigLIP) [Tschannen+ 2025]
- Interleaved MRoPE: distribute all axes (temporal, width, height) to low- and high-frequency bands ... [t w h t w h t w h t w h] rather than [t t t t w w w w h h h h]
- Add explicit video timestamps (as separate tokens rather in positional embeddings)
- Square-root-normalized per-token loss: balance text and multimodal data (video examples are long, don't want to dominate)

> 视觉编码器： SigLIP-2（与 SigLIP 架构相同） [Tschannen+ 2025] Interleaved MRoPE：把所有轴（时间、宽、高）分配到低频与高频段 ... [t w h t w h t w h t w h] 而不是 [t t t t w w w w h h h h] 加入显式的视频时间戳（作为独立的 token，而不是放进位置编码） 按平方根归一化的 per-token loss：平衡文本与多模态数据（视频样本很长，不想让它主导）

Adapter:

- DeepStack: cross-layer fusion to inject visual information into multiple layers [Meng+ 2024]

> 适配器： DeepStack：跨层融合，把视觉信息注入多个层 [Meng+ 2024]

Training:

- Pre-training has 4 stages (train adapter, train all parameters on 8K, 32K, 256K lengths)

Stage	Objective	Training	Token Budget	Sequence Length
S0	Vision-Language Alignment	Merger	67B	8,192
S1	Multimodal Pre-Training	All	~1T	8,192
S2	Long-Context Pre-Training	All	~1T	32,768
S3	Ultra-Long-Context Adaptation	All	100B	262,144

> 图：Qwen3-VL 预训练各阶段表：S0 视觉-语言对齐（仅训练 Merger，67B token，序列长 8192）；S1 多模态预训练（全参数，约 1T token）；S2 长序列扩展（8K→32K→262K）等阶段，并列出配套的数据工程（图像-文本对、OCR、图表、交错多模态数据等）。

- Post-training: SFT on long CoT data, knowledge distillation, RL

> 训练： 预训练有 4 个阶段（训练 adapter；在 8K、32K、256K 长度上训练全部参数） 后训练：在长 CoT 数据上做 SFT、知识蒸馏、RL

	Benchmark	Qwen3-VL 235B-A22B		Gemini 2.5 Pro		OpenAI GPT-5		Claude Opus 4.1	
		thinking	instruct	thinking	budget-128	high	minimal	thinking	non-thinking
STEM Puzzle	MMMU	80.6	78.7	81.7*	80.9	**84.2***	74.4*	78.4	77.2
	MMMU-Pro	69.3	68.1	68.8*	71.2	**78.4***	62.7*	64.8	60.7
	$\text{MathVista}_{\text{mini}}$	**85.8**	84.9	82.7*	77.7	81.3	50.9	75.5	74.5
	MathVision	**74.6**	66.5	73.3*	66.0	70.9	45.8	64.3	57.7
	$\text{MathVision}_{\text{WP}}$	**63.8**	57.0	63.2	56.9	62.8	40.1	54.0	46.4
	We-Math	74.8	67.5	**80.6**	74.5	73.8	51.8	65.2	60.2
	$\text{MathVerse}_{\text{mini}}$	**85.0**	72.5	82.9	65.9	84.1	43.0	70.6	68.1
	DynaMath	82.8	79.4	80.0	78.5	**85.4**	74.0	75.1	72.0
	Math-VR	**66.8**	65.0	64.7*	54.3	58.1	21.7	54.3	38.0
	ZeroBench	**4**	2	3	1	2	2	3	1
	VlmsAreBlind	79.5	80.4	**86.1**	78.5	80.5	53.4	77.8	72.2
	LogicVista	**72.2**	65.8	72.0	68.7	71.8	46.3	67.3	63.5
	VisuLogic	**34.4**	29.9	31.6	26.9	28.5	27.2	27.9	27.2
	VisualPuzzles	57.2	54.7	**60.9**	56.9	57.3	47.9	48.8	47.6
General VQA	MMBench-EN	88.8	89.3	**90.1***	88.4	83.8	81.3	79.4	83.0
	MMBench-CN	88.6	88.9	**89.7***	86.4	83.5	79.9	84.9	74.3
	RealWorldQA	81.3	79.2	78.0*	76.0	**82.8**	77.3	69.9	68.5
	MMStar	**78.7**	78.4	77.5*	78.5	76.4	65.2	72.1	71.0
	SimpleVQA	61.3	63.0	**65.4**	66.9	61.8	56.7	56.7	55.7
Alignment	HallusionBench	**66.7**	63.2	63.7*	60.9	65.7	53.7	60.4	55.1
	MM-MT-Bench	**8.5**	8.5	8.4*	7.6	7.6	7.5	7.8	7.9
	MIA-Bench	**92.7**	91.3	92.3	91.3	92.4	92.6	91.2	90.0
Document Understanding	DocVQA_{test}	**96.5**	97.1	92.6	94.0	91.5	89.6	92.5	89.2
	InfoVQA_{test}	**89.5**	89.2	84.2	82.9	79.0	69.9	69.4	60.9
	$\text{AI2D}_{\text{w. M.}}$	89.2	89.7	**90.9**	90.0	89.7	84.1	86.4	84.4
	ChartQA_{test}	**90.3**	90.3	83.3	62.6	59.7	59.1	86.2	83.9
	OCRBench	**875**	920	866	872	810	787	764	750
	$\text{OCRBench_v2}_{\text{en}}$	**66.8**	67.1	54.3	55.2	53.0	48.2	48.4	47.2
	$\text{OCRBench_v2}_{\text{zh}}$	**63.5**	61.8	48.5	53.1	43.2	37.7	43.7	38.0
	CC-OCR	**81.5**	82.2	77.2	76.8	68.3	66.1	69.1	66.0
	$\text{OmniDocBench}_{\text{en}}$	**0.155**	0.143	0.347	0.206	0.356	0.174	0.194	-
	$\text{OmniDocBench}_{\text{zh}}$	**0.207**	0.207	0.238	0.249	0.472	0.389	0.293	-
	CharXiv(DQ)	90.5	89.4	**94.4**	87.8	89.2	79.5	88.5	87.8
	CharXiv(RQ)	66.1	62.1	67.9	62.9	**81.1***	57.8	63.6	60.2
	$\text{MMLongBench}_{\text{Doc}}$	**56.2**	57.0	55.6	51.2	51.5	42.4	54.5	48.1
2D/3D Grounding	RefCOCO-avg	**92.1**	91.9	74.6*	-	66.8	-	-	-
	CountBench	**93.7**	93.0	91.0*	91.0	91.7	87.8	93.1	91.9
	ODinW-13	**43.2**	48.6	33.7*	34.5	-	-	-	-
	ARKitScenes	**53.7**	56.9	-	-	-	-	-	-
	Hypersim	**11.0**	13.0	-	-	-	-	-	-
	SUNRGBD	**34.9**	39.4	29.7	-	-	-	-	-
Embodied/Spatial Understanding	ERQA	52.5	51.3	55.3	50.3	**65.7***	42.0*	34.8	28.0
	VSI-Bench	**60.0**	62.7	-	-	-	-	-	-
	EmbSpatialBench	**84.3**	83.1	79.1	73.3	82.9	75.1	69.2	66.0
	RefSpatialBench	**69.9**	65.5	36.5	35.6	23.8	23.1	-	-
	RoboSpatialHome	**73.9**	69.4	47.5	49.2	53.5	43.6	-	-
Multi-Image	BLINK	67.1	70.7	70.6*	70.0	**71.0**	62.8	64.1	62.9
	MUIRBENCH	**80.1**	73.0	77.2	74.0	77.5	66.5	-	-
Video Understanding	MVBench	75.2	76.5	69.9	65.8	**75.3**	64.6	61.4	59.0
	$\text{Video-MME}_{\text{w/o sub.}}$	79.0	79.2	**85.1**	80.6	84.7	77.3	75.6	73.3
	$\text{MLVU}_{\text{M-Avg}}$	83.8	84.3	85.6	81.2	**86.2**	78.3	73.5	71.2
	LVBench	63.6	67.7	**73.0**	69.0	-	-	-	-
	$\text{Charades-STA}_{\text{mIoU}}$	**63.5**	64.8	-	-	-	-	-	-
	VideoMMMU	80.0	74.7	83.6*	79.4	**84.6***	61.6*	76.2	70.1
	MMVU	71.1	68.1	**74.9**	72.2	73.0	68.1	66.4	61.4
Perception with Tool	V*	**85.9**	93.7+	83.8	72.7	72.8	56.7	-	-
	HRBench4K	84.3	85.4+	**87.3**	84.8	-	-	-	-
	HRBench8K	76.6	82.4+	**85.4**	80.1	-	-	-	-
Multi-Modal Coding	Design2Code	**93.4**	92.0	89.2	90.3	92.5	88.9	88.5	85.3
	ChartMimic	78.4	80.5	83.9	79.9	62.1	41.4	**85.2**	82.9
	UniSVG	65.8	69.8	70.0	67.9	71.7	74.5	**73.0**	72.5
Multi-Modal Agent	ScreenSpot Pro	**61.8**	62.0	-	-	-	-	-	-
	OSWorldG	**68.3**	66.7	45.2	-	-	-	-	-
	AndroidWorld	**62.0**	63.7	-	-	-	-	-	-
	OSWorld	**38.1**	31.6	-	-	-	-	-	44.4
	WindowsAA	**32.1**	28.9	-	-	-	-	-	-

> 图：Qwen3-VL 评测结果表：比较 Qwen3-VL 235B-A22B（thinking/instruct）、Gemini 2.5 Pro（thinking, budget=128）、OpenAI GPT-5（high/minimal）与 Claude Opus 4.1（thinking/non-thinking）在 15 大类基准上的得分；类别涵盖 STEM 谜题（MMMU、MathVista 等）、通用视觉问答（MMBench、RealWorldQA、MMStar）、文档理解（DocVQA、OCRBench）、2D/3D 定位、具身与空间理解、多图、视频理解、工具感知、多模态编码与多模态智能体（OSWorld 等）。

Summary:

- SOTA performance
- Lots of data work, but not many details
- Minor but potentially important architectural improvements
- Scale up

> 总结： SOTA 性能 数据工作上投入很大，但细节不多 一些次要但可能重要的架构改进 扩大规模

Chameleon [Chameleon Team 2024]

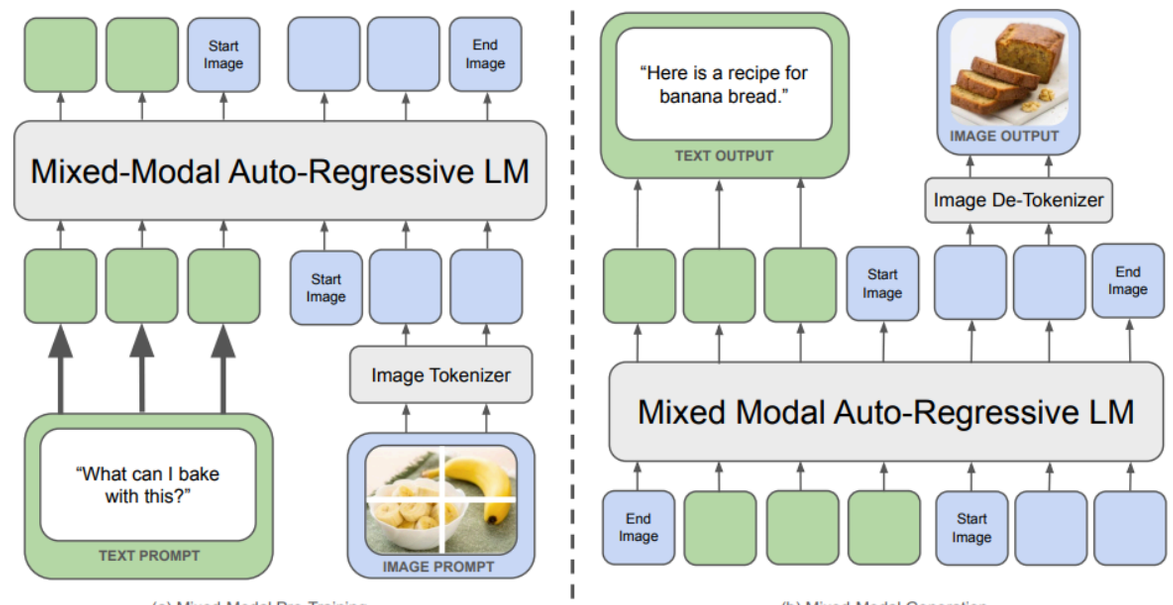
> Chameleon [Chameleon Team 2024]

So far: VLMs encode images (via CLIP or SigLIP), inject into LM

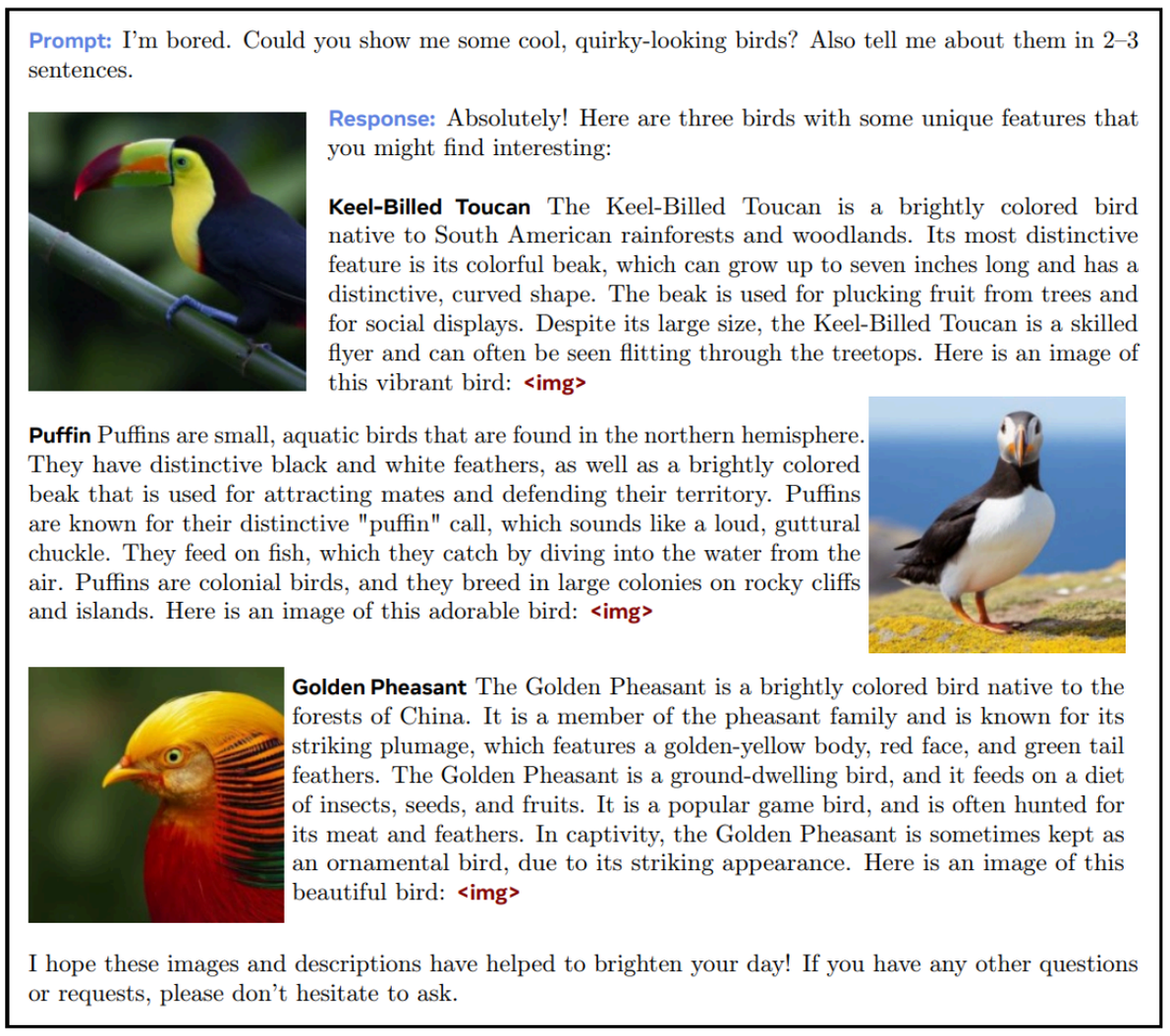
Disadvantage: can't generate images (need diffusion)

> 目前为止：VLM（通过 CLIP 或 SigLIP）编码图像并注入 LM 缺点：无法生成图像（需要 diffusion）

Chameleon: map everything into discrete tokens
Advantage: can analyze and generate images in a uniform way



> 图：Chameleon 早期融合统一模型示意——图像与文本统一被离散化为 token（图像经 VQ tokenizer 编码），全部送入同一个 Transformer，自回归地交替生成文本与图像 token，实现统一建模。


Prompt: I'm bored. Could you show me some cool, quirky-looking birds? Also tell me about them in 2–3 sentences.

Response: Absolutely! Here are three birds with some unique features that you might find interesting:

Keel-Billed Toucan The Keel-Billed Toucan is a brightly colored bird native to South American rainforests and woodlands. Its most distinctive feature is its colorful beak, which can grow up to seven inches long and has a distinctive, curved shape. The beak is used for plucking fruit from trees and for social displays. Despite its large size, the Keel-Billed Toucan is a skilled flyer and can often be seen flitting through the treetops. Here is an image of this vibrant bird: **<img>**

Puffin Puffins are small, aquatic birds that are found in the northern hemisphere. They have distinctive black and white feathers, as well as a brightly colored beak that is used for attracting mates and defending their territory. Puffins are known for their distinctive "puffin" call, which sounds like a loud, guttural chuckle. They feed on fish, which they catch by diving into the water from the air. Puffins are colonial birds, and they breed in large colonies on rocky cliffs and islands. Here is an image of this adorable bird: **<img>**

Golden Pheasant The Golden Pheasant is a brightly colored bird native to the forests of China. It is a member of the pheasant family and is known for its striking plumage, which features a golden-yellow body, red face, and green tail feathers. The Golden Pheasant is a ground-dwelling bird, and it feeds on a diet of insects, seeds, and fruits. It is a popular game bird, and is often hunted for its meat and feathers. In captivity, the Golden Pheasant is sometimes kept as an ornamental bird, due to its striking appearance. Here is an image of this beautiful bird: **<img>**

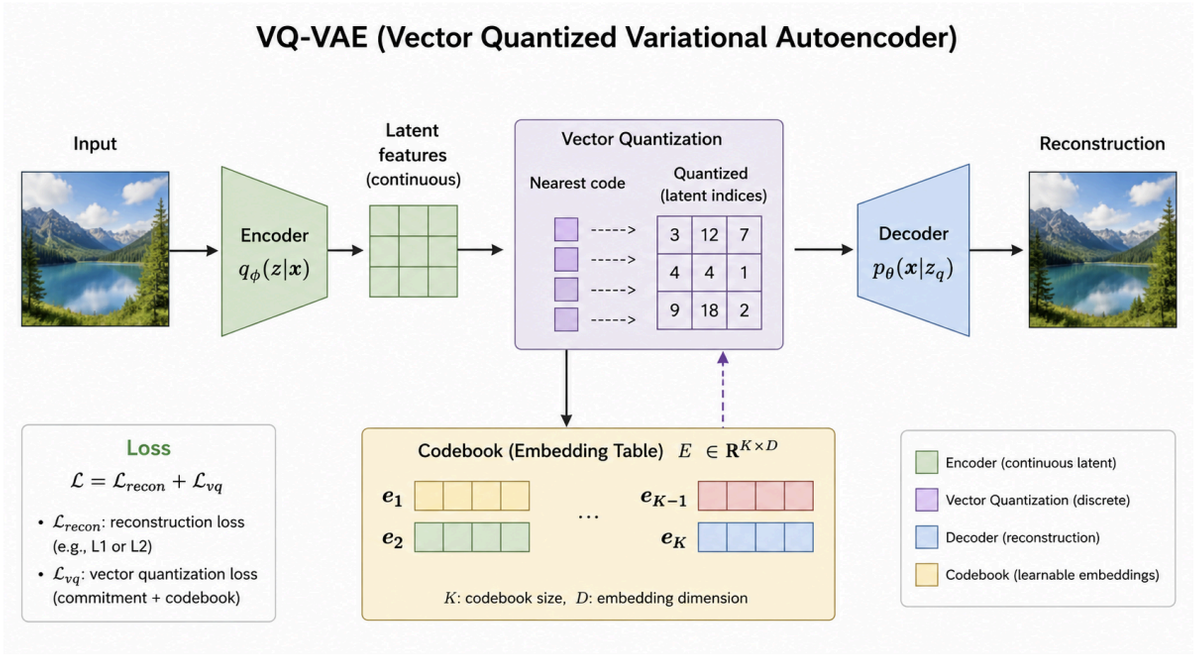
I hope these images and descriptions have helped to brighten your day! If you have any other questions or requests, please don't hesitate to ask.

> 图：Chameleon 早期融合多模态示例：提示为"I'm bored. Could you show me some cool, quirky-looking birds? Also tell me about them in 2–3 sentences."，模型以文字与图像交错生成回答，介绍三种奇特的鸟。

> Chameleon：把一切都映射成离散 token 优点：能以统一的方式分析和生成图像

Vision encoder [Gafni+ 2022]

- Key difference: encoder needs to map to discrete tokens (so we can generate them)
- VQ-VAE (Vector Quantized Variational Autoencoder) [Oord+ 2017]
- Idea: map image to a discrete codebook, decode back to image and minimize reconstruction loss



图：VQ-VAE（向量量化变分自编码器）：输入经编码器 q_ϕ(z|x) 得到连续潜在特征，通过向量量化在码本中查找最近邻编码，得到量化后的潜在索引（如 3 12 7 / 4 4 1 / 9 2 5）；解码器据此重建输出，训练使用直通估计器（straight-through estimator），损失包含 L1/MSE 重构项与 KL 散度（先验 N(0,1)）。

- Encodes 512 x 512 image into 1024 tokens (codebook of size 8192)
- Train a new BPE tokenizer

> 视觉编码器 [Gafni+ 2022] 关键区别：编码器需要映射到离散 token（这样我们才能生成它们） VQ-VAE（向量量化变分自编码器） [Oord+ 2017] 思路：把图像映射到离散码本，再解码回图像，并最小化重构损失 把 512 x 512 的图像编码成 1024 个 token（码本大小 8192） 训练一个新的 BPE tokenizer

Training:

- Stage 1 (80%): large-scale, unsupervised (2.9T text tokens, 1.5T text/image tokens, 400B text/image interleaved tokens)
- Stage 2 (20%): 50% of stage 1 data, 50% of high quality data

> 训练： 阶段 1（80%）：大规模、无监督（2.9T 文本 token，1.5T 文本/图像 token，400B 图文交错 token） 阶段 2（20%）：50% 用阶段 1 的数据，50% 用高质量数据

Training stability

- Text tokens have low entropy, image tokens have high entropy, leads to norm growth, logit drift problem
- Fixes: QK norm, z-loss regularization

> 训练稳定性 文本 token 熵低，图像 token 熵高，会导致范数增长、logit 漂移问题 修复手段：QK norm、z-loss 正则化

Summary:

- Elegant (just autoregressive modeling of discrete tokens)
- Not as performant (discretization loses information - think OCR)
- Training with multiple modalities is tricky

> 总结： 优雅（只是对离散 token 做自回归建模） 性能没那么好（离散化会损失信息——想想 OCR） 多模态一起训练并不容易